İLERİ GÖMÜLÜ SİSTEMLER

LABORATUVAR 2 RAPORU

GPIO Erişim Yöntemleri, Buton Kesmesi ve Zamanlayıcılar

Adı Soyadı	Gamze Şirin	**Öğrenci No**	2510140022
Öğretim Üyesi	Dr. Murat Sürücü	**Tarih**	25.10.2026

1. Deneyin Amacı

Bu deneyde ESP32-S3 üzerinde bir GPIO pininin registerlarla ve ESP-IDF sürücüsüyle nasıl ayarlandığı karşılaştırılmış, üç farklı erişim yönteminin hızı ölçülmüş, BOOT düğmesi kesmesi ile kontak sıçraması incelenmiş ve GPTimer ile esp_timer zamanlayıcılarının Wi-Fi yükü altındaki kararlılığı karşılaştırılmıştır.

2. Kullanılan Donanım ve Yazılım

- ESP32-S3-N16R8 geliştirme kartı (işlemci saati 240 MHz), USB-C kablo, breadboard, erkek-erkek jumper kablo
- Windows bilgisayar, ESP-IDF v6.1, `idf.py` monitörü, `xtensa-esp32s3-elf-objdump`
- Föyde verilen projeler: `lab2_gpio_hiz`, `lab2_buton`, `lab2_zamanlayici`; histogram için `hist_ciz.py`

3. Ölçüm Yöntemi

Bu dönem lojik analizör kullanılmadığı için ölçümler kartın kendi sayaçlarıyla yapılmıştır. Süreler 240 MHz'deki işlemci çevrim sayacıyla (`esp_cpu_get_cycle_count()`, 1 çevrim = 4,17 ns) ölçülmüş, çıktılar monitörde Ctrl+T, Ctrl+L ile dosyaya kaydedilmiştir. Her kayıttan önce kart Ctrl+T, Ctrl+R ile yeniden başlatılarak açılışta yazılan register değerleri de kayda alınmıştır. Görev 3'te kenar zamanları kesme işleyicisinde damgalandığı için, kesme gecikmesinden kısa aralıklarla gelen kenarlar tek kesmede birleşebilir; bu nedenle konsoldaki kenar sayıları gerçek değerin alt sınırıdır. Ham kayıtlar depoda `lab2/olcum/` klasöründedir.

4. Görev 1: Register ile Ayar ve gpio_config()

Yapılan işlem

Program GPIO4'ü önce `gpio_cikis_ayarla()` ile doğrudan registerlara yazarak, sonra `gpio_config()` ile yeniden ayarlamış ve her iki durumda dört registeri konsola yazdırmıştır. Föyün istediği deneme için kod `.mode = GPIO_MODE_INPUT_OUTPUT` ile ikinci kez derlenip çalıştırılmıştır.

Ölçüm sonuçları

IO_MUX_GPIO4 alanı	Register ile ayar	gpio_config() OUTPUT	gpio_config() INPUT_OUTPUT
MCU_SEL [14:12] (pin işlevi)	1 (GPIO)	1	1
FUN_DRV [11:10] (sürme gücü)	2	2	2
FUN_IE [9] (giriş tamponu)	1	**0**	1
FUN_PU [8] / FUN_PD [7]	0 / 0	0 / 0	0 / 0
Uyku alanları [6:0]	0x02	0x02	0x02
Register değeri	`0x00001A02`	`0x00001802`	`0x00001A02`

Tablo 1. IO_MUX_GPIO4 registerinin alanları. GPIO_PIN4 = 0x00000000, GPIO_FUNC4_OUT_SEL_CFG = 0x00000500 (OUT_SEL = 256, OEN_SEL = 1) ve GPIO_ENABLE'ın 4. biti = 1 üç durumda da aynıdır.

Değerlendirme

Register ile yapılan ayar giriş tamponunu açık bırakmaktadır. `gpio_config()` ise `GPIO_MODE_OUTPUT` kipini "yalnız çıkış" olarak uygular ve girişi kapatır. Pin `INPUT_OUTPUT` olarak ayarlandığında giriş açık kaldığından iki değer aynı çıkmıştır. Çıkış yolu (OUT_SEL, ENABLE) her durumda aynı olduğu için LED'in davranışı değişmez; değişen tek şey pinin düzeyinin `GPIO_IN` registerından okunup okunamayacağıdır.

5. Görev 2: Erişim Yöntemlerinin Hızı

Yapılan işlem

GPIO4 her yöntemle, kesmeler kapalıyken 10 000 kez 1 ve 0 yapılmış; toplam süre çevrim sayacıyla ölçülüp periyot başına çevrime bölünmüştür. Ölçüm yaklaşık 1,4 s'de bir tekrarlandığından kayıtta her yöntem için 21–22 ölçüm bulunmaktadır. Ardından derlenmiş kod `objdump` ile incelenmiştir.

Ölçüm sonuçları

Yöntem	Ölçüm sayısı	Çevrim / periyot	Frekans (Hz)	Döngüdeki komut	Periyottaki erişim
Sürücü API'si (`gpio_set_level`)	22	102 (hepsi)	2 352 519	8 (2'si `call8`)	2 × `call8`
Yapı eşlemesi (`GPIO.out_w1ts`)	22	29 (21 kez), 30 (1 kez)	8 000 773	8	2 × (`memw` + `s32i.n`)
Ham register (`REG(...)`)	21	29 (hepsi)	8 000 773	9	2 × (`l32r` + `memw` + `s32i.n`)

Tablo 2. Periyot süreleri. Frekanstan hesaplanan değerler 102,02 ve 29,997 çevrimdir, konsol tamsayıya yuvarlar. Açılıştan sonraki ilk ölçüm biraz daha yavaştır (sürücüde 2 345 000 Hz, yapıda 30 çevrim).

degistir_surucu (döngü)
```
movi    a11, 1
movi    a10, 4
call8   gpio_set_level
movi.n  a11, 0
movi.n  a10, 4
call8   gpio_set_level
addi.n  a2, a2, 1
bltu    a2, a7, döngü
```

degistir_yapi (döngü)
```
l32r    a9, GPIO (0x60004000)
movi.n  a10, 16
memw
s32i.n  a10, a9, 8   ; out_w1ts
memw
s32i.n  a10, a9, 12  ; out_w1tc
addi.n  a8, a8, 1
bltu    a8, a2, döngü
```

degistir_register (döngü)
```
movi.n  a9, 16
l32r    a10, 0x60004008
memw
s32i.n  a9, a10, 0
l32r    a10, 0x6000400C
memw
s32i.n  a9, a10, 0
addi.n  a8, a8, 1
bltu    a8, a2, döngü
```

Şekil 1. `objdump` çıktısından üç fonksiyonun döngü gövdesi (`lab2/olcum/g2_disasm.txt`).

Değerlendirme

Sürücü API'si diğer iki yöntemden yaklaşık 3,5 kat yavaştır, çünkü her periyotta iki fonksiyon çağrısı yapılmaktadır. Yapı eşlemesi ve ham register döngüleri 8 ve 9 komuttan oluşmasına rağmen ikisi de 29–30 çevrim sürmüştür; süreyi komut sayısı değil, `memw` ile beklenen iki çevre birimi yazması belirlemektedir. İlk ölçümün yavaş olması, fonksiyonların flash'ta bulunmasıyla uyumludur: kod ilk çalıştırmada önbelleğe yüklenir, sonraki ölçümler önbellekten çalışır.

6. Görev 3: Buton Kesmesi ve Sıçrama

Yapılan işlem

Açılışta `IO_MUX_GPIO0 = 0x00001B02` (MCU_SEL = 1, FUN_IE = 1, FUN_PU = 1) ve `GPIO_PIN0 = 0x00002180` (INT_TYPE = 3, iki kenar; CPU0 kesme izni) okunmuştur. Kesme CPU0'ın 8. hattına atanmış, matris eşlemesi de 8 çıkmıştır; föyde beklendiği gibi iki değer aynıdır. `SESSIZLIK_MS = 20` ile önce BOOT düğmesine 20 kez basılmış (yavaş, normal, hızlı), ardından GND'ye bağlı bir kablo GPIO0 satırına 10 kez değdirilmiştir. Deneme daha sonra 1 ms ve 50 ms ile tekrarlanmıştır.

Ölçüm sonuçları

Basış	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Basılı süre (ms)	200	220	250	220	210	20	190	200	190	200	180	200	190	190	160	40	200	170	210	160
BIRAKMA kenar / µs	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	**5/1 111**	1/0	1/0	1/0	1/0

Tablo 3. BOOT düğmesiyle 20 basış (`SESSIZLIK_MS = 20`). BASMA olaylarının hepsi tek kenar ve 0 µs'dir. Basılı süre, BASMA ve BIRAKMA zaman damgaları arasındaki farktır (10 ms çözünürlük). 2. ve 3. basış arasında 16,7 ms arayla gelen 2 kenarlık bir "gürültü" grubu da kaydedilmiştir. 16. basışın bırakmasındaki kenarlar: 0, 8, 97, 151, 1 111 µs.

Temas	1	2	3	4	5	6	7	8	9	10	11
BASMA kenar	3 349	544	179	794	2	1	378	1 410	1	1	339
BASMA süre (µs)	129 207	43 339	15 764	44 427	34	0	6 818	42 984	0	0	25 064
BIRAKMA kenar	2 373	2 556	874	685	767	124	507	605	292	341	562
BIRAKMA süre (µs)	60 358	119 902	26 793	26 481	7 603	12 407	12 912	12 854	7 224	12 338	17 208
Kayıp (birikimli)	0	80	80	80	106	106	118	432	456	480	480

Tablo 4. Kablo temasıyla `SESSIZLIK_MS = 20`. 10 temas yapılmış, program 11 basış saymıştır. Bunların dışında 14 066 kenar içeren 32 "gürültü" grubu oluşmuş, en uzunu 200,8 ms sürmüştür.

SESSIZLIK_MS	Gerçek bekleme (100 Hz tık)	BOOT yapılan → sayılan	Kablo temas yapılan → sayılan	Gürültü grubu (kablo)	Sayılan kenar (kablo)	Kayıp kenar
1	0 tık (beklemez)	5 → 5	5 → **8**	30	1 569	**4 443**
20	2 tık (10–20 ms)	20 → 20	10 → 11	32	30 750	480
50	5 tık (40–50 ms)	10 → 10	5 → **5**	1	2 251	**0**

Tablo 5. Sessizlik süresinin etkisi. 50 ms denemesindeki BOOT basışlarının 5'i kısa tutulmuş, en kısası 150 ms sürmüştür. Temasların şiddeti denemeden denemeye değiştiği için kenar sayıları yalnızca niteliksel olarak karşılaştırılabilir.

Değerlendirme

BOOT düğmesi neredeyse hiç sıçramamıştır; bu, düğme hattında kartta bir süzme olabileceğini düşündürmektedir (kart şeması görülmediği için doğrulanmamıştır). Yine de 40 ms'lik çok kısa bir basışın bırakılmasında 1,1 ms'lik bir sıçrama görülmüştür. Kablo teması ise kötü bir kontağa benzer şekilde yüzlerce, bazen binlerce kenar ve onlarca milisaniyelik kararsızlık üretmiştir. 1 ms denemesinde birçok grubun tam 64 kenarda kesilmesi, 64 elemanlık kuyruğun dolduğunu göstermektedir. 1 ms süzgecin çalışmamasının nedeni FreeRTOS'un 100 Hz tık hızıdır: `pdMS_TO_TICKS(1)` 0 tık eder ve `xQueueReceive` hiç beklemez, bu yüzden tek temas birden çok basış sayılmıştır.

7. Görev 4: GPTimer ve esp_timer

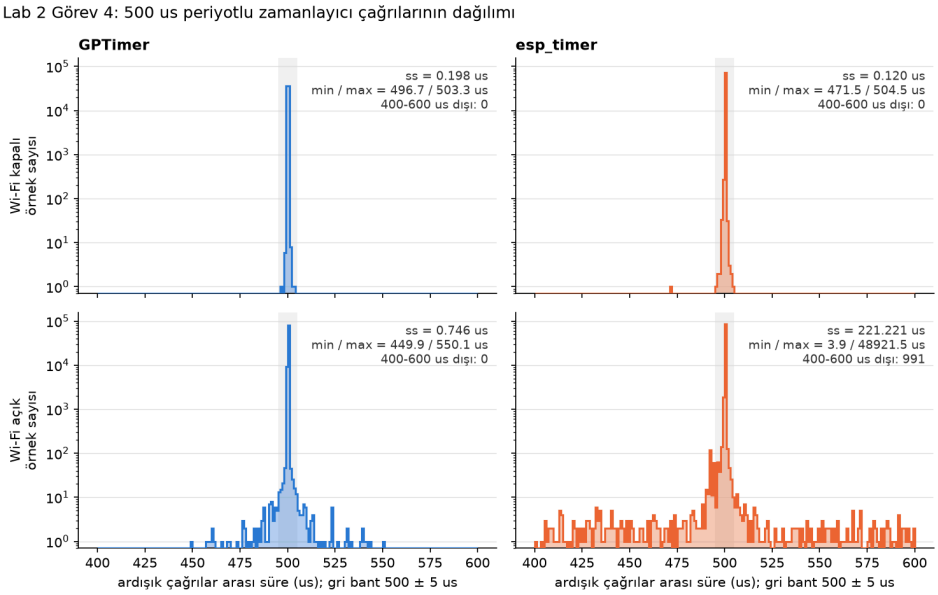
Yapılan işlem

Açılış registerları iki kayıtta da aynıdır: `TIMG0_T0CONFIG = 0xE00A0400` (EN, INCREASE, AUTORELOAD ve ALARM_EN = 1; DIVIDER = 80, yani 80 MHz APB'den 1 MHz sayım; USE_XTAL = 0), `T0ALARMLO = 500`, `T0LOADLO = 0`, `INT_ENA = 0x5`, `TG0_T0` kesme eşlemesi = 9. İki zamanlayıcının 500 µs'lik çağrıları arasındaki süre çevrim sayacıyla ölçülmüş ve 2 s'lik pencereler halinde özetlenmiştir. Kayıt önce Wi-Fi kapalı (18 pencere), sonra `WIFI_YUKU = 1` ile Wi-Fi açık (22 pencere) alınmıştır.

Ölçüm sonuçları

Durum	Örnek	ss, tüm kayıt (µs)	ss, kararlı (µs)	En küçük / en büyük, kararlı (µs)	495–504 µs içinde, kararlı	<400 / >600 µs kararlı
GPTimer, Wi-Fi kapalı	72 516	0,198	0,197	498,34 / 502,04	%100,00	0 / 0
esp_timer, Wi-Fi kapalı	72 517	0,120	0,048	496,88 / 503,08	%100,00	0 / 0
GPTimer, Wi-Fi açık	91 216	0,746	0,677	449,94 / 550,08	%99,89	0 / 0
esp_timer, Wi-Fi açık	91 264	221,2	44,46	4,08 / 1 589,55	%98,53	446 / 316

Tablo 6. Bütün durumlarda ortalama 500,000 µs'dir. "Kararlı" değerler, resetten sonraki ilk 2 s'lik pencere çıkarılarak hesaplanmıştır. Wi-Fi açık kayıtta bu pencere Wi-Fi'nin başlatıldığı anı içerir ve esp_timer'da 48 921 µs'lik tek bir aralık vardır.



Şekil 2. Ardışık çağrılar arasındaki sürenin dağılımı (logaritmik eksen; gri bant 500 ± 5 µs), `hist_ciz.py` ile çizilmiştir.

Değerlendirme

Wi-Fi kapalıyken iki zamanlayıcı da kararlı durumda 500 µs'nin ±3,1 µs yakınında kalmıştır. GPTimer örneklerinin 499 ve 500 µs kutularına yarı yarıya bölünmesi gerçek bir salınım değildir; kutular tamsayıya aşağı yuvarlandığı için 500,000 µs'nin hemen altındaki ve üstündeki değerler iki kutuya ayrılmaktadır. Wi-Fi açıkken esp_timer'da 1,5 ms'ye varan gecikmelerin hemen ardından 4–7 µs'lik çok kısa aralıklar gelmektedir: gecikmiş çağrılar arka arkaya çalışıp açığı kapattığı için ortalama 500 µs'de kalmakta, ancak tek tek periyotlar bozulmaktadır.

8. Soruların Cevapları

Soru 1. Görev 1'de gpio_config() sonrası değişen bit(ler) hangileridir ve neden değişmiştir?

Yalnızca IO_MUX_GPIO4'ün 9. biti (FUN_IE) 1'den 0'a inmiştir. `gpio_cikis_ayarla()` giriş tamponunu açık bırakırken `gpio_config()`, `.mode = GPIO_MODE_OUTPUT` kipinde girişi kapatır. Kip `GPIO_MODE_INPUT_OUTPUT` yapıldığında FUN_IE 1 kalmış ve iki register değeri de 0x00001A02 olmuştur; tahmin doğrulanmıştır.

Soru 2. Sürücü API'si ile ham register arasındaki fark nedir? Yapı eşlemesi ile ham register neden aynı hızdadır?

Sürücü döngüsü her periyotta `gpio_set_level`'i iki kez `call8` ile çağırır. Fonksiyon `entry` ile yeni bir register penceresi açar, pin numarasının geçerli ve çıkış olabilen bir pin olduğunu kaydırma ve maske işlemleriyle denetler (`bltu`, `ssl/sll`, `ssr/srl`, `movnez`, `bbsi`), GPIO adresini iki `l32i` ile okur, düzeye ve pinin 32'den küçük olmasına göre dallanır ve ancak bundan sonra `memw` + `s32i.n` ile yazıp `retw` ile döner. Bu, çağrı başına yaklaşık 30 komut eder ve periyodu 102 çevrime çıkarır. Yapı eşlemesinde `GPIO.out_w1ts` derleme anında sabit bir adres+ofsete (0x60004000 + 8) çözülür; `REG(GPIO_OUT_W1TS_REG)` de aynı adrese (0x60004008) gider. İkisi de iki `memw` + `s32i.n` çiftine derlenir, aradaki tek fark ham sürümdeki fazladan bir `l32r`'dir. Süreyi yazma sayısı belirlediği için ikisi de 29–30 çevrimde kalır.

Soru 3. Her register yazmasından önce gelen memw komutunun görevi nedir ve neden derleyici tarafından eklenmiştir?

`memw`, Xtensa'da bir bellek bariyeridir: kendisinden önceki tüm yükleme ve yazmalar tamamlanmadan sonraki bellek erişimi başlamaz. GPIO registerları `volatile` olarak tanımlandığı için derleyici (Xtensa GCC'nin varsayılan `-mserialize-volatile` davranışı) her volatile erişimden önce `memw` ekler. Böylece W1TS ve W1TC yazmaları birleşmeden veya yer değiştirmeden program sırasıyla çevre birimine ulaşır. Bedeli her yazmada beklemedir; Görev 2'deki 29 çevrimin büyük kısmı bu beklemeden oluşur.

Soru 4. Kesme işleyicisi neden GPIO_STATUS_W1TC registerına yazmak zorundadır? Bu satır kaldırılırsa ne olur?

GPIO kesme durumu GPIO_STATUS'ta bir bayrak olarak tutulur ve CPU'ya düzey olarak iletilir. Bayrak yazılımla silinmedikçe kesme hattı etkin kalır. İşleyici W1TC'ye yazmazsa, işleyiciden dönülür dönülmez aynı kesme yeniden alınır; işlemci sürekli işleyiciye girip çıkar, görevler çalışamaz ve sonunda kesme bekçisi (interrupt watchdog) kartı resetler.

Soru 5. Ölçülen en uzun sıçrama süresine göre güvenli bir SESSIZLIK_MS değeri nedir? Büyük seçilmesinin kullanıcı açısından maliyeti nedir?

BOOT düğmesinde ölçülen en uzun sıçrama 1,1 ms'dir. Kablo temasında tek bir grup 129 ms'ye çıkmıştır, ancak bu bir düğmeden çok kötü bir kontağa özgüdür. Gerçek bir mekanik düğme için 20 ms (100 Hz tık hızında fiilen 10–20 ms) yeterli pay bırakır. Kalitesi bilinmeyen bir giriş için 50 ms önerilir: kablo denemesinde bile her temas tek basış sayılmış ve kenar kaybı olmamıştır. Değeri büyütmenin bedeli tepki gecikmesidir, çünkü olay son kenardan SESSIZLIK_MS sonra işlenir. Ayrıca bu süreden kısa bir bas-bırak dizisi tek grupta birleşir ve "gürültü" sayılarak kaybolur; saniyedeki en yüksek basış sayısı da yaklaşık 1/(2·SESSIZLIK_MS) ile sınırlanır. 1 ms gibi tık süresinin altındaki değerler 0 tık eder ve süzgeci tamamen kapatır.

Soru 6. GPTimer ile esp_timer arasındaki standart sapma farkı nasıl açıklanır? Wi-Fi yükü hangi kaynağı daha çok etkilemiştir?

GPTimer'da TIMG0 alarmı doğrudan bir CPU kesmesi üretir ve geri çağırma IRAM'deki kesme işleyicisinde çalışır; tek gecikme kaynağı kesme gecikmesi ve kesmelerin kapalı tutulduğu kısa bölümlerdir. esp_timer'da ise SYSTIMER alarmı önce bir kesme üretir, bu kesme `esp_timer` görevini (öncelik 22, CPU0) uyandırır ve geri çağırma o görevde çalışır; görev zamanlayıcısı ek bir belirsizlik getirir. Wi-Fi sürücü görevi CPU0'da 23 önceliğiyle çalışır (`wifi driver task … prio:23, core=0`) ve tarama sırasında sık sık devreye girdiği için esp_timer görevi sırada bekler. Bu nedenle Wi-Fi, esp_timer'ın kararlı durum standart sapmasını yaklaşık 900 kat (0,048 → 44,5 µs), GPTimer'ınkini ise yalnızca 3,4 kat (0,197 → 0,677 µs) artırmıştır. Wi-Fi'den en çok etkilenen esp_timer olmuştur.

Soru 7. 500 µs periyotlu ve en fazla ±5 µs sapmalı bir motor kontrol döngüsü için hangi zamanlayıcı seçilmelidir?

GPTimer seçilmelidir. Wi-Fi kapalıyken örneklerin %100'ü 495–504 µs aralığında kalmış, en büyük sapma 3,3 µs olmuştur. Wi-Fi açıkken de kararlı durumda örneklerin %99,89'u bu aralıktadır. esp_timer'da ise Wi-Fi açıkken kararlı durumda bile 762 örnek 400–600 µs dışına çıkmış ve en büyük sapma yaklaşık 1,1 ms olmuştur. Ancak GPTimer'da da Wi-Fi açıkken ±50 µs'lik tekil sapmalar görüldüğünden, kesin ±5 µs garantisi için ek önlem gerekir: Wi-Fi'yi diğer çekirdeğe almak, işleyiciyi IRAM'de ve yüksek öncelikte tutmak ya da asıl çıkışı MCPWM gibi bir donanım biriminde üretmek.

9. Sonuç

1. `gpio_config()`, `GPIO_MODE_OUTPUT` ile yalnızca giriş tamponu bitini (FUN_IE) kapatmıştır; `INPUT_OUTPUT` ile iki ayar aynı çıkmıştır.
2. Bir periyot sürücü API'siyle 102 çevrim (2,35 MHz), yapı eşlemesi ve ham registerla 29–30 çevrim (8,0 MHz) sürmüştür. Fark `gpio_set_level` içindeki denetimlerden gelmektedir.
3. BOOT düğmesinde 40 olayın 39'u tek kenardır; yalnızca çok kısa bir basışın bırakılmasında 1,1 ms'lik sıçrama görülmüştür. Kablo temasında sıçrama 129 ms'ye kadar çıkmıştır.
4. `SESSIZLIK_MS = 1` süzgeci fiilen kapatmış (5 temas 8 basış sayılmış), 50 ms'de 5 temasın hepsi doğru sayılmış ve hiç kenar kaybolmamıştır.
5. Wi-Fi yükü esp_timer'ın kararlılığını belirgin şekilde bozmuş, GPTimer'ı çok az etkilemiştir. Hassas periyotlu işler için GPTimer uygundur.

Not: Yapay zekâdan yararlanılmıştır.